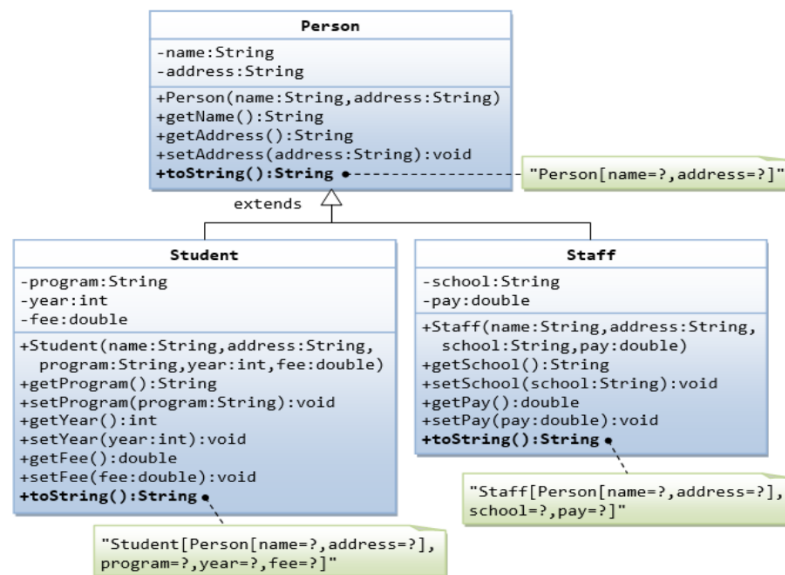


Assignment

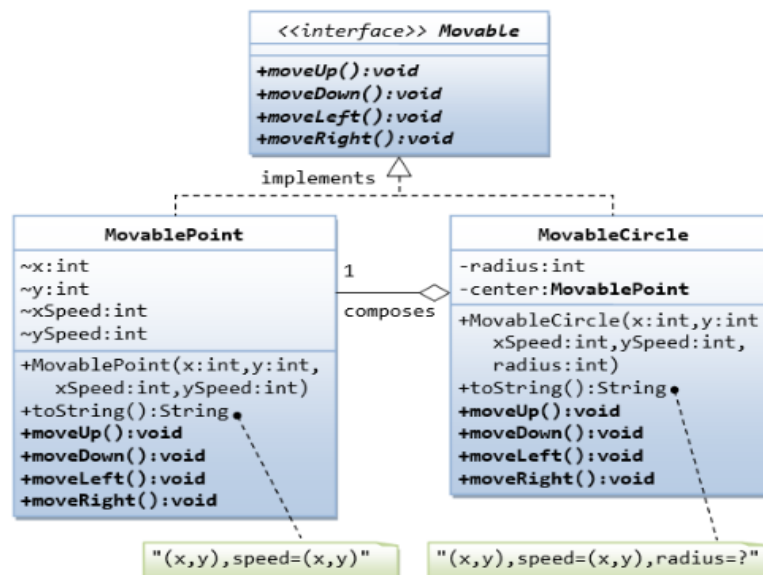
Assignment # : 11

Topic : Inheritance, Polymorphism and Abstraction

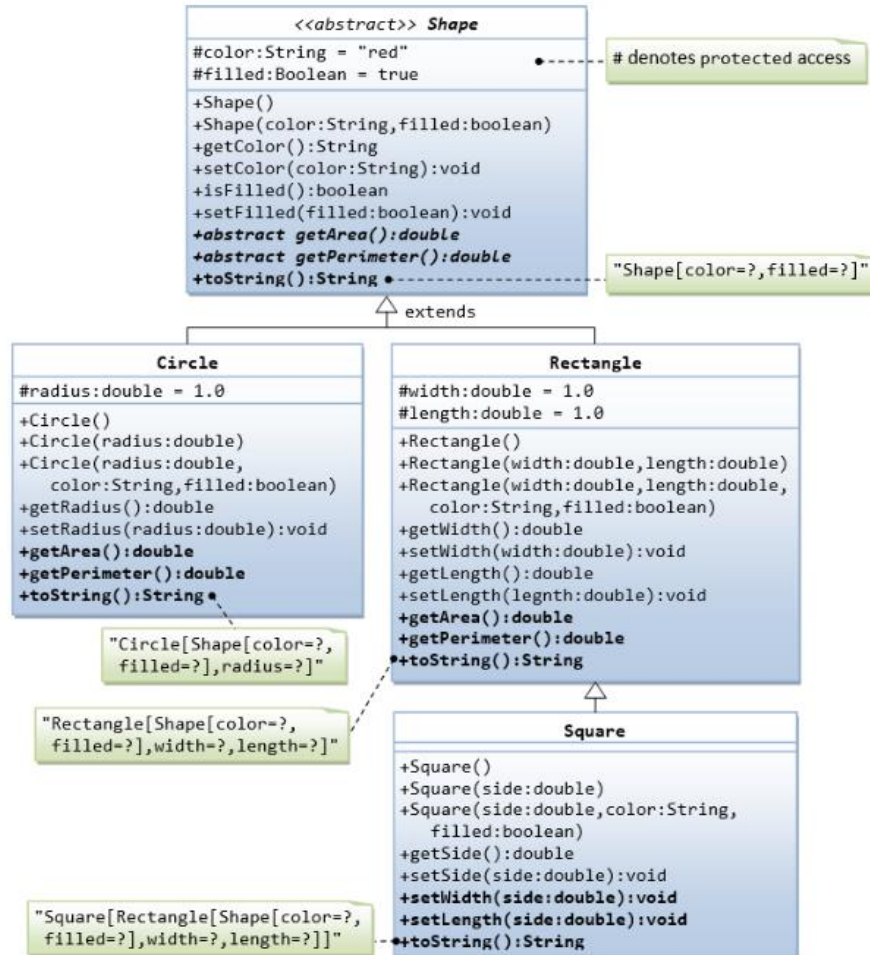
1. Implement the following class hierarchy and display the details of student and staff Object.



2. Write an interface called Movable, which contains abstract methods moveUp(), moveDown(), moveLeft() and moveRight(), as shown in the class diagram. Also write the implementation classes called MovablePoint and MovableCircle. Test the classes for the implemented classes.



3. Implement the below calss hierarchy and test the classes for methods implemented.



4. Banking:

All the banks operating in India are controlled by RBI. RBI has set a well-defined guideline (e.g. minimum interest rate, minimum balance allowed, maximum withdrawal limit etc) which all banks must follow. For example, suppose RBI has set minimum interest rate applicable to a saving bank account to be 4% annually; however, banks are free to use 4% interest rate or to set any rates above it.

Write a JAVA program to implement bank functionality in the above scenario and demonstrate the dynamic polymorphism concept. Note: Create few classes namely Customer, Account, RBI (Base Class) and few derived classes (SBI, ICICI, PNB etc). Assume and implement required member variables and functions in each class.

Hint:

Class Customer

```
{  
  //Personal Details ...  
  // Few functions ...  
}
```

Class Account

```
{  
  // Account Detail ...  
  // Few functions ...  
}
```

Class RBI

```
{  
  Customer c; //hasA relationship  
  Account a; //hasA relationship  
  ..  
  Public double GetInterestRate() { }  
  Public double GetWithdrawalLimit() { }  
}
```

Class SBI extends class RBI

```
{  
  //Use RBI functionality or define own functionality.  
}
```

Class ICICI extends RBI

```
{  
  //Use RBI functionality or define own functionality.  
}
```

5. Painting Shapes:

Develop a class hierarchy of shapes and write a program that computes the amount of paint needed to paint different objects. The hierarchy will consist of a parent class Shape with three derived classes - Sphere, Rectangle, and Cylinder. For the purposes of this exercise, the only attribute a shape will have is a name and the method of interest will be one that computes the area of the shape (surface area in the case of three-dimensional shapes). Do the following.

1. Write an abstract class Shape with the following properties:
 - An instance variable shapeName of type String
 - An abstract method area()
 - A toString method that returns the name of the shape
2. The file Sphere.java contains a class for a sphere which is a descendant of Shape. A sphere has a radius and its area (surface area) is given by the formula $4 \cdot \pi \cdot \text{radius}^2$. Define similar classes for a rectangle and a cylinder. Both the Rectangle class and the Cylinder class are descendants of the Shape class. A rectangle is defined by its length and width and its area is length times width. A cylinder is defined by a radius and height and its area (surface area) is $\pi \cdot \text{radius}^2 \cdot \text{height}$. Define the toString method in a way similar to that for the Sphere class.
3. The file Paint.java contains a class for a type of paint (which has a "coverage" and a method to compute the amount of paint needed to paint a shape). Correct the return statement in the amount method so the correct amount will be returned. Use the fact that the amount of paint needed is the area of the shape divided by the coverage for the paint. (NOTE: Leave the print statement - it is there for illustration purposes, so you can see the method operating on different types of Shape objects.)
4. The file PaintThings.java contains a program that computes the amount of paint needed to paint various shapes. A paint object has been instantiated. Add the following to complete the program:
 - Instantiate the three shape objects: deck to be a 20 by 35 foot rectangle, bigBall to be a sphere of radius 15, and tank to be a cylinder of radius 10 and height 30.
 - Make the appropriate method calls to assign the correct values to the three amount variables.
 - Run the program and test it. You should see polymorphism in action as the amount method computes the amount of paint for various shapes